Project: P2P Payment Ledger

Simplified UPI / Wallet

Background

Build a peer-to-peer digital wallet application where users can register, load money, transfer money to other users, and track their transactions.

All transfers follow an ACH-style delayed settlement model — money leaves the sender immediately but is credited to the receiver only after a 3-business-day holding period. The system should detect basic fraud patterns, notify users of important events, run background settlement and housekeeping jobs, and provide an admin view for monitoring.

Users can also create multiple simulated bank accounts with manually maintained balances. These accounts can be used to add money to their wallet manually or through an Auto Reload feature. No real bank integration or actual financial transactions are involved.

Timeline: 1 week

Tech Stack: Spring Boot 3.x, Spring Data JPA, Spring Security (JWT), Spring Scheduler, PostgreSQL, React.

Functional Requirements

1. User Registration & Authentication

- A user registers with name, email, phone number, password, and a 4-digit transaction PIN.
- On successful registration, the system automatically creates a wallet for the user with ₹0 balance.
- Users log in using email and password. The system issues a JWT token on successful login.
- The system tracks failed login attempts. After 5 consecutive failed attempts, the account is locked for 30 minutes.
- There are two roles — USER and ADMIN. Regular users can only access their own data. Admins have access to the admin panel.
- The transaction PIN is separate from the login password and is required for every money transfer.

2. Wallet

- Every user has exactly one wallet, created automatically during registration with ACTIVE status.
- Users can add money to their wallet by selecting one of their simulated bank accounts. The selected account is debited and the wallet is credited by the same amount.
- The wallet must display the available balance — money the user can spend or transfer right now.
- The wallet must display today's total spending.
- Users can enable Auto Reload to replenish their wallet automatically using a selected simulated bank account when the wallet balance falls below a configured threshold.
- Wallet funding and Auto Reload are simulated operations. No real bank integration is required.

Wallet statuses:

- **Active:** The wallet is fully functional. The user can send and receive money and use Auto Reload.
- **Inactive:** The user has self-deactivated their wallet. All outgoing transactions are blocked. Incoming settlements are declined, the transaction is marked DECLINED, the sender's available balance is refunded, and the sender is notified. Auto Reload is also disabled while the wallet is inactive. The user can reactivate the wallet at any time.
- **Frozen:** Only an admin can freeze or unfreeze a wallet. All outgoing transactions are blocked. Incoming settlements are declined, the transaction is marked DECLINED, the sender is refunded, and the sender is notified. Auto Reload is blocked while the wallet is frozen.

When a user tries to send money from an Inactive or Frozen wallet, the transaction must be rejected with a clear error message indicating the wallet status.

When a pending settlement is due for a receiver whose wallet is Inactive or Frozen, the settlement job must decline the transaction, refund the sender, and notify the sender.

3. Simulated Bank Accounts

Users can create and manage multiple simulated bank accounts. These accounts represent funding sources for demonstration purposes and are not connected to real banks.

Account management:

- Users can create multiple simulated accounts.
- Each account contains a bank name, account nickname, generated demo account identifier, and manually entered initial simulated balance.
- Users can view their accounts and current simulated balances.

- Users can add additional demo funds to an account through an **Add Demo Funds** operation.
- Demo funding is a simulation only. It does not represent a real deposit or verified bank balance.
- Users can select any eligible simulated bank account when manually adding money to their wallet or configuring Auto Reload.
- Users can deactivate an account when it is not required, provided it is not being used in an active operation.
- Users can access and modify only their own simulated accounts.
- Account balances must never become negative.
- Simulated account balance changes must be traceable, including the amount, operation type, timestamp, and outcome.

Manual wallet top-up:

1. The user selects a simulated bank account.
2. The user enters the amount to add to the wallet.
3. The system validates that the amount is positive and the selected account has sufficient simulated funds.
4. The system deducts the amount from the simulated account and credits the wallet.
5. Both balance changes are committed atomically. If either operation fails, neither change is committed.
6. The operation is recorded in the transaction history and ledger, and the user receives a notification.

4. Auto Reload Wallet

Users can configure an automatic wallet top-up using one of their simulated bank accounts.

Configuration:

- Users can turn Auto Reload ON or OFF.
- Users select one eligible simulated bank account as the funding source.
- Users configure a minimum wallet balance threshold.
- Users configure a fixed reload amount.
- Users can update their settings or disable Auto Reload at any time.
- Auto Reload is available only while the wallet is ACTIVE.

Automatic reload behavior:

- Whenever a successful wallet balance change causes the available balance to fall below the configured threshold, the system attempts an automatic reload.
- The system checks whether the selected simulated bank account has sufficient funds.
- If sufficient funds are available, the configured reload amount is deducted from the simulated account and credited to the wallet.

- If the simulated account has insufficient funds, the reload fails, neither balance is changed, and the user is notified.
- If the wallet is Inactive or Frozen, Auto Reload must not execute.
- Auto Reload must not create duplicate reloads when multiple transactions or concurrent balance changes occur.
- A failed reload must not affect the outcome of an otherwise successful payment or wallet operation.
- A successful reload must be recorded in transaction history and the ledger using a distinct transaction type, such as AUTO_RELOAD.
- The user receives an in-app notification for successful and failed reload attempts.
- Auto Reload must not trigger recursively because of its own wallet credit. The implementation should prevent repeated reloads from the same trigger.

Example:

- Current wallet balance: ₹400
- Configured threshold: ₹500
- Fixed reload amount: ₹2,000
- Selected simulated account balance: ₹10,000

Since ₹400 is below ₹500, the system attempts to reload ₹2,000. If successful, the wallet balance becomes ₹2,400 and the simulated account balance becomes ₹8,000.

If the selected account contains only ₹1,000, the reload fails and both balances remain unchanged.

5. Send Money (ACH — Delayed Settlement)

- A user can search for another registered user by phone number or email.
- To send money, the user selects a recipient, enters an amount, adds an optional remark, and confirms with their 4-digit transaction PIN.

When a transfer is initiated:

- The sender's available balance is reduced immediately.
- A DEBIT ledger entry is created for the sender.
- The receiver's wallet is not credited immediately. The transfer amount remains an outstanding settlement obligation in the system.
- The transaction status is set to PENDING.
- The transaction records a `settlement_due_date` calculated as three business days from initiation.

Only the sender can see the pending transfer in their transaction history with status PENDING and the expected settlement date. The receiver is not notified until the transfer is settled and credited.

A background settlement job processes eligible transfers and credits the receiver's available balance once the settlement date is reached.

On settlement:

- The receiver's available balance is increased by the amount.
- A CREDIT ledger entry is created for the receiver.
- The transaction status changes from PENDING to SUCCESS.
- Both sender and receiver are notified.

If a transfer fails during settlement, the transaction is marked FAILED, the sender's available balance is refunded, and the sender is notified.

Each payment request must carry a unique `request_id` generated by the client. If the same `request_id` is submitted again, the system returns the original result rather than processing the payment again.

6. Fraud Controls

Every send-money request must pass through the following checks before processing.

Daily transaction limit

A user cannot send more than ₹50,000 in a single day. If the transaction would exceed this limit, it must be blocked and a fraud alert must be created.

Duplicate transaction detection

If a user tries to send the exact same amount to the exact same recipient within two minutes of a previous transaction, it must be blocked as a likely duplicate. A fraud alert must be created.

High-value transaction flag

If a single transaction exceeds ₹8,000, it is allowed to proceed normally and enters PENDING settlement, but a fraud alert must be created for admin review.

The admin has until the settlement date to review the alert and decide:

- **Block:** The transaction is marked FAILED, the sender's available balance is refunded, and both parties are notified.
- **Dismiss:** The alert is dismissed, and the transaction remains pending until its scheduled settlement date.

Alerts for transactions that have already settled are read-only.

All fraud alerts must appear in a queue visible to admins.

7. Transaction History

Users must be able to view a mini-statement showing the last 10 transactions and their full transaction history with pagination and filters for:

- Date range
- Type: sent, received, wallet top-up, Auto Reload, or all
- Status: success, pending, failed, declined, or all

Pending transfers must clearly show the current status, expected settlement date, and days remaining until settlement.

Clicking on a transaction must show full details, including associated ledger entries. For pending transfers, only the sender's DEBIT entry exists until settlement.

Users must be able to see their personal stats: total sent, total received, and this month's transaction count.

Wallet top-ups and Auto Reload operations must appear in transaction history and be distinguishable from P2P transfers.

8. Notifications

The system must generate in-app notifications for important events.

Users must see an unread count on a bell icon in the header, view all notifications, and mark them as read.

Event	Who is notified	Example
Transfer initiated	Sender	Transfer initiated with expected settlement date
Transfer settled	Sender	Transfer has been settled
Transfer settled	Receiver	Amount received from sender

Transfer failed	Sender	Transfer failed and amount refunded
Transaction blocked	Sender	Transaction blocked due to daily limit
Fraud alert raised	Admin	High-value transaction flagged for review
Account locked	User	Account locked after multiple failed logins
Account unlocked	User	Account automatically unlocked
Wallet top-up successful	User	Wallet successfully topped up from selected simulated account
Auto Reload successful	User	Wallet automatically reloaded with the configured amount
Auto Reload failed	User	Auto Reload failed due to insufficient simulated funds or another validation failure

Demo funds added	User	Simulated funds successfully added to the selected account

9. Scheduled Background Jobs

The following jobs must run automatically on a schedule with no manual trigger.

ACH Settlement Processor — Every hour

Finds all transactions where status is PENDING and `settlement_due_date` is on or before today.

For each eligible transaction:

- Credits the receiver's available balance.
- Creates the CREDIT ledger entry.
- Updates the transaction status to SUCCESS.
- Sends settlement notifications to both sender and receiver.
- If an individual settlement fails, marks that transaction FAILED, refunds the sender, and continues processing the remaining transactions.

Each settlement batch must be logged with the number of transactions settled, number failed, and total amount settled.

Daily Limit Reset — Midnight

Resets daily spending counters so users can transact again the next day.

Account Auto-Unlock — Every 5 minutes

Finds accounts locked for more than 30 minutes, unlocks them, and sends an account-unlocked notification to each affected user.

Daily Summary & Reconciliation — 11 PM

Aggregates the day's activity, including:

- Total number of transfers initiated and their total amount.
- Total number of transfers settled and their total amount.
- Number of fraud alerts raised.
- Number of new user registrations.
- Total pending settlement amount across all users.

Also performs a reconciliation check for settled transactions, comparing the sum of DEBIT ledger entries against the sum of CREDIT ledger entries. Pending transfers are excluded from this check. Any mismatch generates a critical admin notification.

Every scheduled job run must be logged with start time, end time, success/failure status, records processed, and error message if any.

Auto Reload is triggered by eligible wallet balance changes rather than relying solely on a periodic scheduler. Its operations must be safe against concurrent requests and retries.

10. Admin Panel

Users with the ADMIN role must have access to an admin section with the following features.

Dashboard

- Total registered users.
- Today's transfers initiated count.
- Today's transfers settled count.
- Today's transaction volume.
- Total pending settlement amount.
- Pending fraud alerts count.
- A line chart showing daily transaction volume over the last 30 days.

User List

- View all users with their wallet balance, wallet status, and registration date.
- View a user's recent transactions.
- Freeze or unfreeze a user's wallet.

Transaction Monitoring

- View all transactions across the system with filters for date, status, and amount range.
- View full transaction details and ledger entries.
- Clearly distinguish pending transactions, wallet top-ups, and Auto Reload operations.

Settlement Monitor

- View all pending transactions with expected settlement dates.
- Show how many are due today, due tomorrow, and overdue.
- Allow an admin to manually trigger settlement for a specific eligible pending transaction when needed.

Fraud Review

- View all pending fraud alerts.

- Block or dismiss eligible high-value transaction alerts.
- Display alerts for already settled transactions as read-only.

Job Monitor

- View the last 20 scheduled job runs with their status and statistics.
- Show settlement counts, failed counts, and total amounts per settlement batch.

Reconciliation

- Display today's total debits versus total credits for settled transactions.
- Show pending settlement amounts separately.
- Flag mismatches.

All admin actions must be written to an audit log.

Business Rules

Rule	Value
Daily transaction limit per user	₹50,000
Duplicate detection window	2 minutes
High-value flag threshold	₹8,000
Settlement period	3 business days
Account lock after failed logins	5 attempts
Auto-unlock after	30 minutes

Initial wallet balance	₹0
Transaction PIN	4 digits
Transaction statuses	PENDING → SUCCESS / FAILED / DECLINED
Roles	USER, ADMIN
Auto Reload threshold	Configurable by user
Auto Reload amount	Fixed amount configurable by user
Simulated bank accounts per user	Multiple
Real bank integration	Not required
Initial simulated account balance	Entered by user as demo funding

Database

The original 11 required tables remain:

1. `users`
2. `login_attempts`
3. `wallets`
4. `transactions`
5. `ledger_entries`

6. `fraud_alerts`
7. `notifications`
8. `audit_log`
9. `scheduled_job_log`
10. `daily_summary`
11. `settlement_batches`

Two new tables are added:

12. `simulated_bank_accounts` — Stores each user's simulated bank name, nickname, demo account identifier, simulated balance, account status, and timestamps.
13. `auto_reload_settings` — Stores the wallet's Auto Reload enabled state, selected simulated account, threshold amount, reload amount, and configuration timestamps.

The `transactions` and `ledger_entries` tables should support the new wallet top-up and Auto Reload operations, with appropriate transaction types and traceable balance changes.

Simulated account balance changes must also be auditable. This can be handled with a dedicated simulated-account transaction history table if required; otherwise, the existing transaction and audit mechanisms must be extended to record these operations reliably.

All balance-changing operations must use database transactions and appropriate locking or conditional updates to prevent negative balances, duplicate reloads, and inconsistent wallet or funding-account balances.